



Universidad Autónoma de Baja California

Programación Orientada a Objetos

Arreglos de objetos

Profesor: MC. Itzel Barriba Cázares

Instancia de un arreglo de objetos

- ▶ Crear una instancia del arreglo de objetos

- ▶ **Sintaxis:**

```
NombreClase arregloObjetos[] = new NombreClase[Array_length];
```

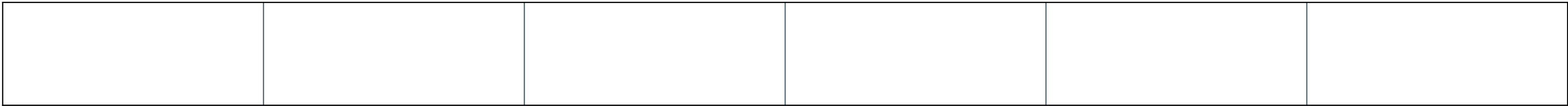
- ▶ **Ejemplo:**

```
Alumno[] arregloAlumnos = Alumno[5];
```

- ▶ Una vez que se crea una instancia de arreglo de objetos de esta manera, los elementos individuales del arreglo de objetos deben crearse utilizando la palabra clave new:

Estructura de un arreglo de objetos

arregloAlumnos



Alumno[0]
Nombre: Lola López
Matricula: 123245

Objeto

Elementos

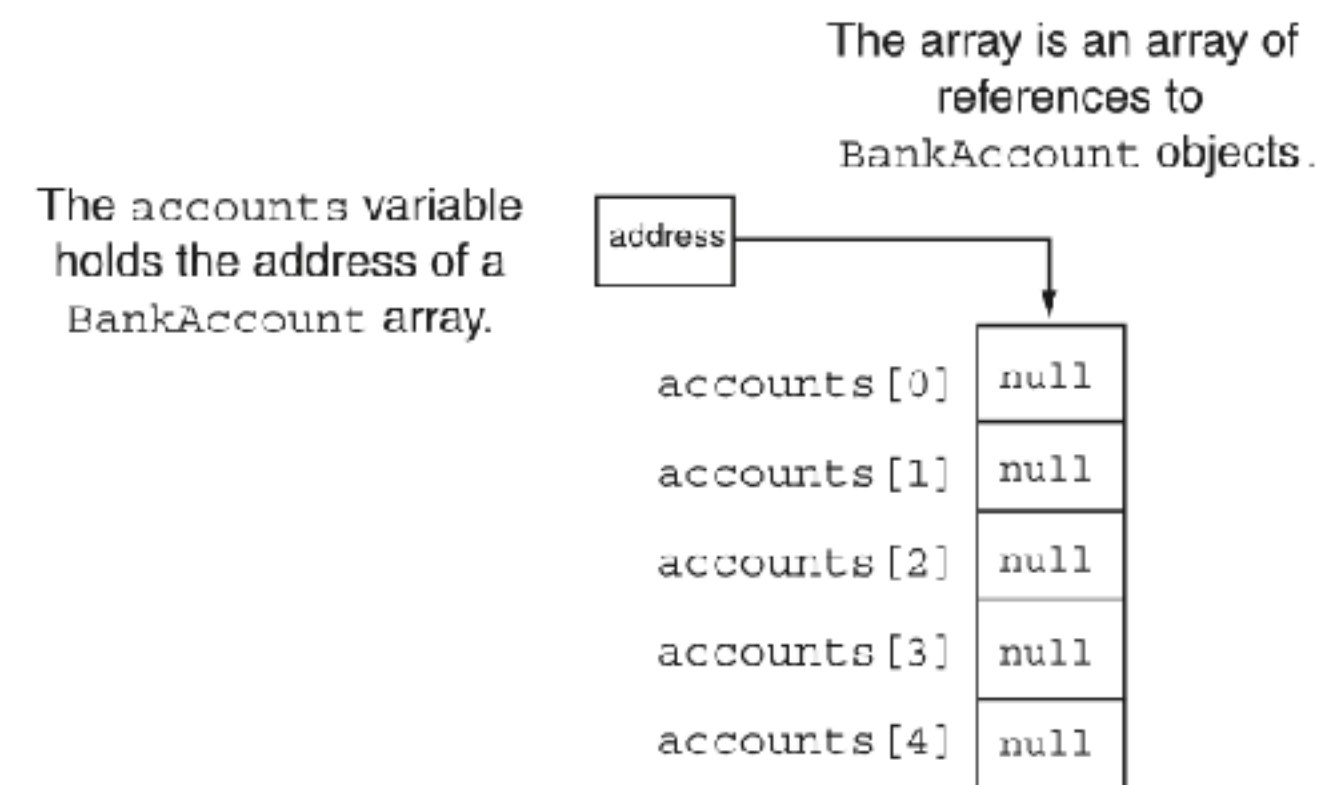
```
Alumno[] arregloAlumnos = Alumno[5];  
arregloAlumno[0] = new Alumno("Lola Lopez",123245);
```


Arreglo de objetos

- Puedes crear arreglos de objetos que sean instancias de las clases. La siguiente instrucción declara un arreglo de cinco objetos de CuentaBancaria.

```
final int NUM_CUENTAS = 5;  
CuentaBancaria[] cuentas = new CuentaBancaria[NUM_CUENTAS];
```

- La variable que hacer referencia al arreglo se llama cuentas. Cada elemento del arreglo es una variable de referencia. Como se muestra:



Arreglo de objetos

- ▶ Debes crear individualmente los objetos a los que hará referencia cada elemento. La siguiente instrucción utiliza un ciclo para crear objetos para cada elemento:

```
for (int index = 0; index < cuentas.length; index++)  
    cuentas[index] = new CuentaBancara();
```

- ▶ Los objetos de un arreglo se acceden mediante subíndices.

```
cuentas[2].setBalance(2500.0);  
cuentas[2].retiro(500.0);
```

Arreglo de objetos

```
/** The BankAccount class simulates a bank  
account. */
```

```
public class BankAccount {  
    private double balance; // Account  
    balance
```

```
    public BankAccount() {  
        balance = 0.0;  
    }
```

```
    public BankAccount(double startBalance) {  
        balance = startBalance;  
    }
```

```
    public BankAccount(String str) {  
        balance = Double.parseDouble(str);  
    }
```

```
    public void deposit(double amount) {
```

```
        public void deposit(String str) {  
            balance += Double.parseDouble(str);  
        }
```

```
        public void withdraw(double amount) {  
            balance -= amount;  
        }
```

```
        public void withdraw(String str) {  
            balance -= Double.parseDouble(str);  
        }
```

```
        public void setBalance(double b) {  
            balance = b;  
        }
```

```
        public void setBalance(String str) {  
            balance = Double.parseDouble(str);  
        }
```

```
        public double getBalance() {  
            return balance;  
        }
```


Arreglo de objetos

```
import java.util.Scanner;
/* This program works with an array of three
BankAccount objects.*/

public class ObjectArray {
    public static void main(String[] args) {
        final int NUM_ACCOUNTS = 3; // Number
of accounts
        BankAccount[] accounts = new
BankAccount[NUM_ACCOUNTS];

        // Create objects for the array.
        createAccounts(accounts);

        // Display the balances of each
account.
        System.out.println("Here are the
balances " + "for each account:");
        for (int index = 0; index <
accounts.length; index++) {
            System.out.println("Account " +
(index + 1) + ": $" +
accounts[index].getBalance());
        }
    }
}
```

```
        private static void
createAccounts(BankAccount[] array) {
            double balance; // To hold an
account balance

            // Create a Scanner object.
            Scanner keyboard = new
Scanner(System.in);
            // Create the accounts.
            for (int index = 0; index <
array.length; index++) {
                // Get the account's
balance.
                System.out.print("Enter
the balance for " + "account " +
(index + 1) + ": ");
                balance =
keyboard.nextDouble();

                // Create the account.
                array[index] = new
BankAccount(balance);
            }
        }
    }
}
```

Inicializar un arreglo de objetos con constructor

- ▶ Usando un constructor
 - ▶ Al momento de crear objetos reales, podemos asignar valores iniciales a cada uno de los objetos pasando valores por separado. Los objetos reales individuales se crean con sus valores distintos.
 - ▶ Ejemplo: se crea un arreglo de objetos de la clase alumno

```
Alumno[] arregloAlumnos = new Alumno[5];  
Alumno[0] = new Alumno("Lola Lopez",123245);  
Alumno[1] = new Alumno("Oscar Martinez",125456);  
Alumno[2] = new Alumno("Maria Garcia",234567);  
Alumno[3] = new Alumno("Juan Sánchez",125459);  
Alumno[4] = new Alumno("Omar Flores",125498);
```


Pase de objetos como argumentos

- ▶ Cuando se pasa un objeto como argumento a un método, **la dirección del objeto se pasa a la variable de parámetro del método**. Es decir, el parámetro hace referencia al objeto.
- ▶ A menudo es necesario escribir métodos que **acepten objetos como argumentos**. Cuando se pasa el objeto como argumento, la variable del parámetro hace referencia al objeto y método tiene acceso al objeto.

Ejemplo pasar un objeto como parámetro

// Este programa incluye un metodo dentro de la clase Box

```
public class Box5{
    double ancho;
    double alto;
    double profundidad;

    Box5(double a, double h, double p)
    {
        ancho = a;
        alto = h;
        profundidad = p;
    }
    Box5(Box5 ob){
        ancho = ob.ancho;
        alto = ob.alto;
        profundidad = ob.profundidad;
    }
}
```

```
Box5() {
    ancho = -1;
    alto = -1;
    profundidad = -1;
}
Box5(double len) {
    ancho = alto = profundidad = len;
}

// Despliega el volumen de un objeto
double volumen() {
    return ancho * alto * profundidad;
}
```

Ejemplo Demo

```
public class DemoCaja5 {  
    public static void main(String[] args) {  
        // declara, aloca e inicializa objetos Caja  
        Box5 miCaja1 = new Box5(10,20,15);  
        Box5 miCaja2 = new Box5();  
        Box5 miCubo = new Box5(7);  
        Box5 miClon = new Box5(miCaja1);  
  
        double vol;  
  
        // obtiene el volumen de la caja1  
        vol = miCaja1.volumen();  
        System.out.println("Volumen de miCaja1 es " + vol);  
  
        // obtiene el volumen de la caja2  
        vol = miCaja2.volumen();  
        System.out.println("Volumen de miCaja2 es " + vol);  
  
        // obtiene el volumen de miCubo  
        vol = miCubo.volumen();  
        System.out.println("Volumen de miCubo es " + vol);  
  
        // obtiene el volumen de miClon  
        vol = miClon.volumen();  
        System.out.println("Volumen de miClon es " + vol);  
    }  
}
```

```
Volumen de miCaja1 es 3000.0  
Volumen de miCaja2 es -1.0  
Volumen de miCubo es 343.0  
Volumen de miClon es 3000.0
```


Ejemplo 2: Clase Rectángulo

```
/*Clase rectangulo*/
public class Rectangle {
    private double length;
    private double width;

    public void setLength(double len) {
        length = len;
    }

    public void setWidth(double w) {
        width = w;
    }

    public double getLength() {
        return length;
    }

    public double getWidth() {
        return width;
    }

    public double getArea() {
        return length * width;
    }
}
```

Pase de objetos como parámetro

```
public class PaseObjetos {  
    public static void main(String [] args){  
        Rectangle box = new Rectangle(12.0, 5.0);  
        displayRectangle(box);  
    }  
    public static void displayRectangle(Rectangle r) {  
        System.out.println("Length : "+r.getLength()+ " Width :"+r.getWidth());  
    }  
}
```

Pase de objetos como parámetro

- ▶ Cuando una variable de referencia se pasa como argumento a un método, el método tiene acceso al objeto al que hace la referencia la variable.
- ▶ Cuando un método recibe una referencia de objeto como argumento, es posible que el método modifique el contenido del objeto al que hace referencia la variable.

Pase de objetos como parámetro

```
public class PaseObjetos2 {  
    public static void main(String[] args) {  
        Rectangle box = new Rectangle(12.0, 5.0);  
        // Display the object's contents.  
        System.out.println("Contents of the box object:");  
        System.out.println("Length : " + box.getLength() + " Width : " + box.getWidth());  
  
        // Pass a reference to the object to the changeRectangle method.  
        changeRectangle(box);  
        // Display the object's contents again.  
        System.out.println("\nNow the contents of the " + "box object are:");  
        System.out.println("Length : " + box.getLength() + " Width : " + box.getWidth());  
    }  
  
    public static void changeRectangle(Rectangle r) {  
        r.setLength(0.0);  
        r.setWidth(0.0);  
    }  
}
```

Copia de objetos

- ▶ Para copiar un objeto en si, debes crear un nuevo objeto y luego establecer los campos del nuevo objeto con los mismos valores que los campos del objeto que se está copiando.
- ▶ Esto se puede simplificar si se escribe un método que realice la operación y devuelva una referencia al objeto duplicado.

Copia de objetos

```
public class Stock
{
    private String symbol;
    private double sharePrice;

    public Stock(String sym, double
price)
    {
        symbol = sym;
        sharePrice = price;
    }

    public String getSymbol()
    {
        return symbol;
    }

    public double getSharePrice()
    {
        return sharePrice;
    }
}
```

```
    public String toString()
    {
        // Create a string describing
the stock.
        String str = "Trading symbol:
" + symbol + "\nShare price: " +
sharePrice;
        // Return the string.
        return str;
    }

    public boolean equals(Stock
object2)
    {
        boolean status;
        if (symbol == object2.symbol
&& sharePrice == object2.sharePrice)
            status = true; // Yes,
the objects are equal.
        else
            status = false;
        return status;
    }

    public Stock copy()
    {
        Stock copyObject = new
Stock(symbol, sharePrice);
        return copyObject;
    }
}
```


Pase de objetos demo

```
public class StockDemo3
{
    public static void main(String[] args)
    {
        // Create a Stock object for the XYZ Company.
        Stock company1 = new Stock("XYZ", 9.62);
        Stock company2;
        company2 = company1.copy();
        // Display the contents of both objects.
        System.out.println("Company 1:\n" + company1);
        System.out.println();
        System.out.println("Company 2:\n" + company2);
        if (company1 == company2)
            System.out.println("Both objects are the same.");
        else
            System.out.println("The objects are different.");
    }
}
```

Método toString()

- ▶ El método toString() normalmente devuelve una cadena que representa el estado de un objeto.
- ▶ El estado de un objeto es simplemente la información que se almacena en los campos del objeto en un momento determinado
- ▶ Crear una cadena que represente el estado de un objeto es una tarea tan común que muchos programadores equipan sus clases con un método que devuelve dicha cadena.

Método toString()

- ▶ Cada clase tiene automáticamente un método toString que devuelve una cadena que contiene el nombre de la clase de objeto, seguido del símbolo @, seguido de un entero que normalmente se basa en la dirección de memoria del objeto.
- ▶ Este método se llama cuando es necesario si no ha proporcionado su propio método toString.

Ejemplo

```
public class Direccion {
    private String calle;
    private int numero;
    private String colonia;
    private int codigoPostal;

    public Direccion(String calle, int numero, String colonia, int codigoPostal) {
        this.calle = calle;
        this.numero = numero;
        this.colonia = colonia;
        this.codigoPostal = codigoPostal;
    }

    public String getCalle() {
        return calle;
    }

    public double getNumero() {
        return numero;
    }

    public String getColonia() {
        return colonia;
    }

    public double getCodigoPostal() {
        return codigoPostal;
    }

    public String toString() {
        // Crea un String describiendo la direccion
        String str = "Direccion: \n" + calle + " #" + numero + " " + colonia + " " + codigoPostal;
        return str;
    }
}
```

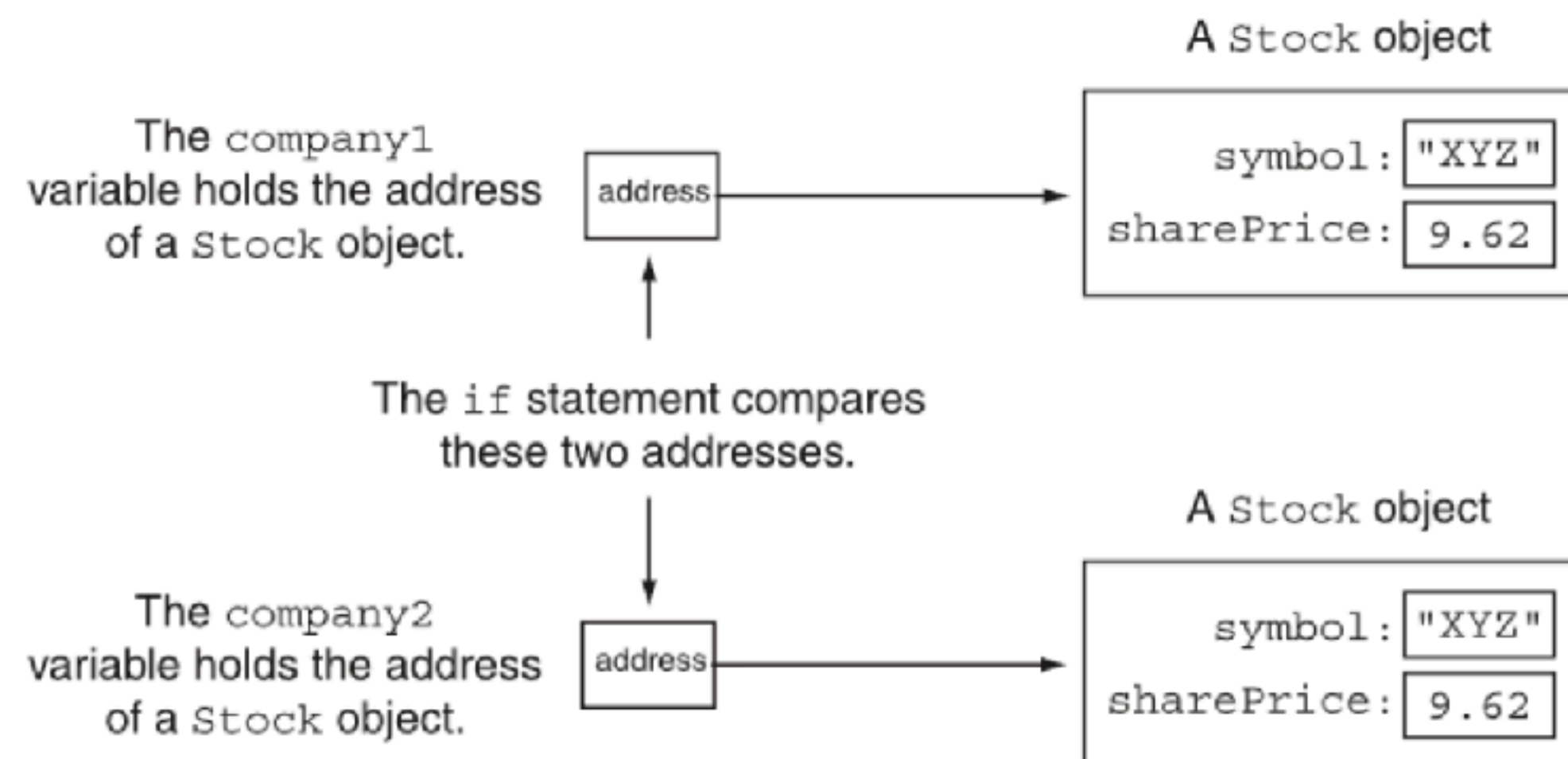
Ejemplo

```
public class DemoDireccion {  
    public static void main(String[] args) {  
        // Crea un objeto Direccion  
        Direccion miDireccion = new Direccion("Calzada tecnologico",  
12023, "Otay", 22190);  
        // Despliega los valores del objeto  
        System.out.println(miDireccion);  
    }  
}
```

Método equals

- No se puede determinar si dos objetos contienen los mismos datos comparándolos con el operador `==`. El **método equals** se utiliza para **comparar el contenido de los objetos**.

```
// Create two Stock objects with the same values.  
Stock company1 = new Stock("XYZ", 9.62);  
Stock company2 = new Stock("XYZ", 9.62);
```



Método equals

- Método que compara el contenido de dos objetos

```
public class Stock {
    private String symbol;
    private double sharePrice;

    public Stock(String sym, double price) {
        symbol = sym;
        sharePrice = price;
    }

    public String getSymbol() {
        return symbol;
    }

    public double getSharePrice() {
        return sharePrice;
    }

    public String toString() {
        // Create a string describing the stock.
        String str = "Trading symbol: " + symbol + "\nShare price: " + sharePrice;
        // Return the string.
        return str;
    }

    public boolean equals(Stock object2) {
        boolean status;
        if (symbol == object2.symbol && sharePrice == object2.sharePrice)
            status = true; // Yes, the objects are equal.
        else
            status = false;
        return status;
    }
}
```

Método equals

- Método que compara el contenido de dos objetos

```
public class StockDemo {  
    public static void main(String[] args) {  
        // Create a Stock object for the XYZ Company.  
        Stock company1 = new Stock("XYZ", 9.62);  
        Stock company2 = new Stock("XYZ", 9.62);  
        if (company1.equals(company2))  
            System.out.println("Both objects are the same.");  
        else  
            System.out.println("The objects are different.");  
    }  
}
```